

Pendulum Data Analysis – Predictions Using PINN and RNN Models

Fábio Luiz Gonçalves Filho
João Pedro da Rosa Mendes
Paul Neugebauer

Abstract

This work evaluates the performance of LSTM-based recurrent neural networks, with and without Physics-Informed Neural Network (PINN) losses, for forecasting the dynamics of simple and double pendulum systems. For the simple pendulum, results show that a standard LSTM achieves lower prediction error and faster convergence than its PINN-augmented counterpart, indicating that the added physical constraints provide no practical benefit in this low-complexity regime. In contrast, the double pendulum—characterized by strong nonlinearity and chaos—shows significant improvements when PINN constraints are applied. Among the tested configurations, a mixed-loss formulation combining motion equations and energy conservation yields the most stable and accurate long-term predictions. Parameter studies further demonstrate the sensitivity of LSTM performance to architectural choices such as batch size, number of layers, and hidden units. Overall, while PINNs offer limited gains for simple dynamics, they provide meaningful improvements in complex chaotic systems when appropriately weighted and tuned.

Contents

1	Introduction	3
2	Data	4
2.1	Simple Pendulum	4
2.1.1	Data Variety	4
2.1.2	The Runge-Kutta Method	5
2.2	Double Pendulum	5
3	Recurrent Neural Network Long Short Term Memory (RNN-LSTM)	5
3.1	Implementation and Parameters	5
3.2	PINN	6
4	Training Process	7
4.1	Simple Pendulum	7
5	Results	9
6	Conclusion	10

1. Introduction

A simple pendulum consists of a mass m attached to a light, rigid rod or string of length l , free to move in a vertical plane. The gravitational force acting on the mass has a tangential component $-mg \sin \theta$, where θ is the angular displacement from the downward vertical. Applying Newton's second law for rotational motion gives the nonlinear equation

$$\ddot{\theta} + \omega_0^2 \sin \theta = 0 \quad \text{with} \quad \omega_0^2 = \frac{g}{l} \quad (1)$$

This equation characterizes the pendulum as a nonlinear oscillatory system whose restoring force depends on $\sin \theta$. Here, ω_0 is the natural angular frequency of the pendulum[1].

Since the restoring term is nonlinear, the resulting differential equation cannot be solved in closed form in terms of elementary functions. This is a general feature of many ordinary differential equations: even for very simple physical systems, the mathematical solutions may involve implicit expressions or special functions, rather than familiar elementary formulas[2]. Due to the simplicity and physical interpretability of the pendulum, it is often taken as a benchmark system to validate numerical solvers, dynamical modeling techniques, and physics-informed machine learning[1].

Based on the simplicity of a pendulum, the double pendulum (two connected pendulums, end to end) introduces an incredible amount of dynamical complexity. Its equations of motion describe a set of coupled nonlinear second-order ordinary differential equations that can be derived through the Lagrangian formalism, rather than Newtonian[3, 2]. Unlike the simple pendulum, the double pendulum is deterministic chaos: the smallest differences in initial conditions result in exponentially divergent trajectories[4]. For this reason, the system is extremely difficult to predict over long time horizons, even using accurate numerical integration. Due to the combination of conceptual simplicity and chaotic behavior, the double pendulum has become one of the most commonly examined examples within nonlinear dynamics.

Taken together, the pendulum and double pendulum provide a natural sequence of increasing complexity, ranging from a well-behaved nonlinear system through to one of the simplest realizations of chaotic motion. This progression provides an ideal testbed for comparing traditional numerical methods with modern computational approaches. A scheme of both, the simple and double pendulum, is given in figure 1.

Recursive Neural Networks (RNN), especially with Long Short Term Memory (LSTM), are well-suited for solving differential equations because they show excellent behavior in modeling time-dependent systems and capturing long-range dependencies, which are common in such equations. Their ability to learn complex nonlinear dynamics makes them ideal for approximating solutions to systems where analytical solutions are difficult or impossible to find. This data-driven approach allows LSTMs to model high-dimensional, dynamic systems, making them a powerful tool in various fields like fluid dynamics, climate modeling, and chaotic systems[6].

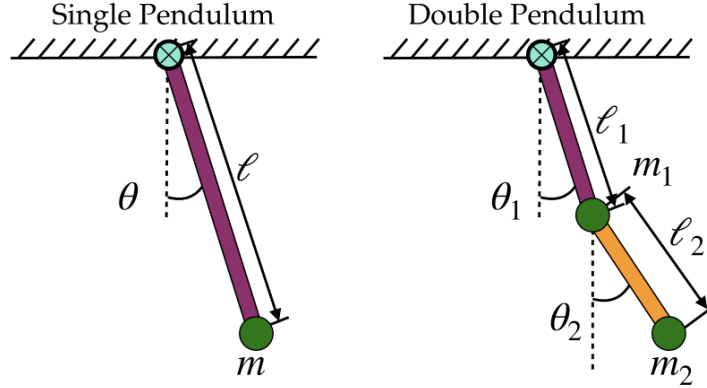


Figure 1: Parametrisation of Single and Double Pendulum, modified from [5]

The focus of this work was made on the simple pendulum case, where the double pendulum was more a testing of the limits of the method and would need closer analysis.

2. Data

2.1. Simple Pendulum

The dataset contains a set of simulated trajectories of a nonlinear simple pendulum. The model includes gravity, linear damping, angular velocity, and its angle variation and length. Each simulation runs for ten seconds at a fixed time step of 0.01s. This yields high-resolution trajectories that are suitable for system identification and learning-based dynamical modeling.

2.1.1. Data Variety

To generate a wide variety of dynamical behaviors, several key parameters are systematically varied. The dataset covers three different values for: gravitational accelerations, lengths of the pendula, damping coefficients, initial angles and initial angular velocities. In total, all combinations of these parameters produce 972 unique simulations that range from nearly harmonic to strongly nonlinear or damped oscillations.

For every simulation, both the angular states and their corresponding Cartesian positions are saved. On top of clean trajectories, four levels of Gaussian noise are added to the Cartesian coordinates, including one noise-free case. Correspondingly, for each of the noise levels, there are two versions of data: (i) a dense, noise-free trajectory with full angle, velocity, and position time series, and (ii) a sparse set of 25 noisy observations that mimic limited real-world sensor measurements.

2.1.2. The Runge-Kutta Method

The Runge-Kutta method (RK4) is commonly used to solve ordinary differential equations, providing accurate approximations by evaluating the system's derivative four times per step and averaging these values. This method is particularly suited for simulating the pendulum system due to its fourth-order accuracy, stability, and ability to handle reasonable timesteps without numerical divergence, making it ideal for generating reproducible reference data for testing learning algorithms and filtering methods [7]. In the dataset creation, each RK4 step calculates four intermediate slopes, which are combined to update the system's state, recording the angle, angular velocity, and position at each timestep. Multiple datasets are generated by varying physical parameters, initial conditions, and measurement noise [7].

2.2. Double Pendulum

The data for the double pendulum [8] was generated by Euler integration, with 0.01 seconds for time step, both pendulums' lengths (in meters) and masses (in kilos) set at 1, gravity at 9.8 (in m/s^2), and randomly generated through a uniform distribution initial angles (with an interval of $[0, 360]$) and angular velocities (with a interval of $[-180, 180]$).

Data was organized within a folder named "datasets", which stores 100 ".csv" files, with 500 steps of a trajectory each.

3. Recurrent Neural Network Long Short Term Memory (RNN-LSTM)

3.1. Implementation and Parameters

The architecture chosen is the Long Short-Term Memory, aka LSTM, a recurrent neural network projected to deal with sequential data, which utilizes a gated memory-cell device to regulate the information flow in and out of the cell, through the lens of three gates: input, output, and forget. This type of neural network aims at mitigating the vanishing/exploding gradients problem, common in vanilla RNNs. Given the brief space for discussion, the main focus of analysis are:

- Batch Size x Number of Epochs
- Hidden Units x Hidden Layers
- Number of iterations
- PINN presence

It's vital to clarify that other parameters will mostly be held at the default values stated on Section 2, unless stated otherwise.

The Long-Short-Term Memory was used as a way to both prevent issues with near-zero gradients and keep data relevancy for longer periods of time.

The model receives as input position-based sequences defined by a window for the time-prediction, and returns an estimation of the next step of the sequence, with or without the addition of a physical-loss, that conditions the model to the solution of a certain physical equation.

Below are the specifications of the model:

- 2 hidden dimensions
- 128 neurons per layer
- Linear output layer
- Adaptive Moment Estimation (ADAM) optimizer
- Reduce on plateau learning rate scheduler

All of the above will be studied in-depth in Section 5 .

3.2. PINN

PINNs are short for Physics-Informed Neural Networks, its central component is the loss function, as it guides the neural network to simultaneously satisfy the constraints imposed by the differential equations of the system, its initial and boundary conditions. In general, as introduced by Raissi, Perdikaris, and Karniadakis [9] the total loss function can be expressed as a sum of different terms:

$$MSE = MSE_u + MSE_f, \quad (2)$$

"The loss MSE_u corresponds to the initial and boundary data while MSE_f enforces the structure imposed by equation (2) at a finite set of collocation points."

The model for the simple case is implemented with the mean-squared error of the residuals from the following differential equation:

$$\ddot{\theta} + \omega_0^2 \sin \theta = 0 \quad \text{with} \quad \omega_0^2 = \frac{g}{l} \quad (3)$$

Concordantly, the double case is implemented with both the mean-squared error of the following motion equations:

$$\begin{aligned} \ddot{\theta}_1 &= \frac{-g(2m_1 + m_2) \sin \theta_1 - m_2 g \sin(\theta_1 - 2\theta_2) - 2 \sin(\theta_1 - \theta_2) m_2 (\dot{\theta}_2^2 l_2 + \dot{\theta}_1^2 l_1 \cos(\theta_1 - \theta_2))}{l_1(2m_1 + m_2 - m_2 \cos(2\theta_1 - 2\theta_2))} \\ \ddot{\theta}_2 &= \frac{2 \sin(\theta_1 - \theta_2) (\dot{\theta}_1^2 l_1 (m_1 + m_2) + g(m_1 + m_2) \cos \theta_1 + \dot{\theta}_2^2 l_2 m_2 \cos(\theta_1 - \theta_2))}{l_2(2m_1 + m_2 - m_2 \cos(2\theta_1 - 2\theta_2))} \end{aligned}$$

And the global energy conservation equation, structured below:

$$E = K + U$$

In which K is the kinetic, and U the potential energies of the system, defined as:

$$K = \frac{1}{2}m_1L_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2\left(L_1^2\dot{\theta}_1^2 + L_2^2\dot{\theta}_2^2 + 2L_1L_2\dot{\theta}_1\dot{\theta}_2\cos(\theta_1 - \theta_2)\right)$$

$$U = -m_1gL_1\cos(\theta_1) - m_2g(L_1\cos(\theta_1) + L_2\cos(\theta_2))$$

In the model, weights for each equation are also set.

4. Training Process

4.1. Simple Pendulum

As this simulation consists of a simplified version of the latter problem, the process was broken down into 2 different sections: the ordinary model, which accepts values for gravity, angular momentum, length, and damping, and the PINN applied one, which also considers a physical loss on it's training.

The graph displayed represents the MSE evolution of both networks trained with a sequence length of 15 for 200 epochs, using an 80/20 train/test split, while applying a learning rate of 0.001 and a regularization of e^{-5} . The physical loss was enforced via the pendulum's differential equation, comparing the predicted and analytical accelerations through a least-squares criterion multiplied by 10^{-16} as its weight.

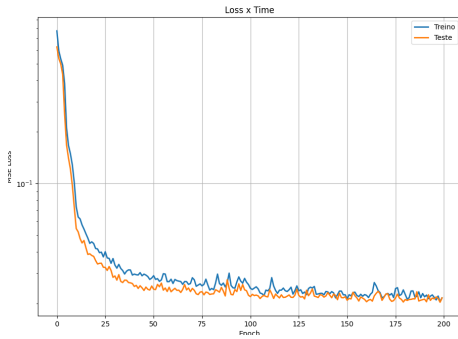


Figure 2: No PINN usage

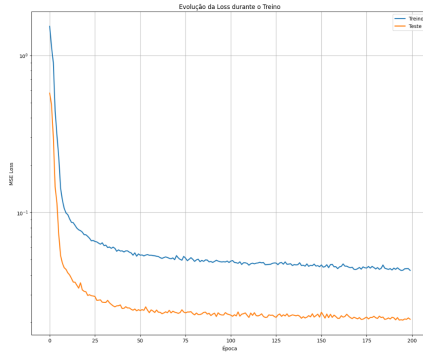


Figure 3: LSTM with PINN

This network was trained using roughly the same parameters as before, but adjusting it's output to accept the sin, cos, angle velocity of both the first and second pendulum, instead of just the former X and Y position on a 2D plane. Given the Chaotic nature of the studied subject, a standard scaler function was used to normalized the angle velocity in order to provide better results. The training process used an 100-step window, regularization of $1e^{-6}$ and learning rate of 0.002.

The physical losses were incorporated based on the equations of motion as well as the momentum conservation law, both derived from the system's Lagrangian. These were enforced through a least-squares criterion and weighted using the following fine-tuned lambda coefficients: 10^{-6} for the motion loss and 1 for the energy loss.

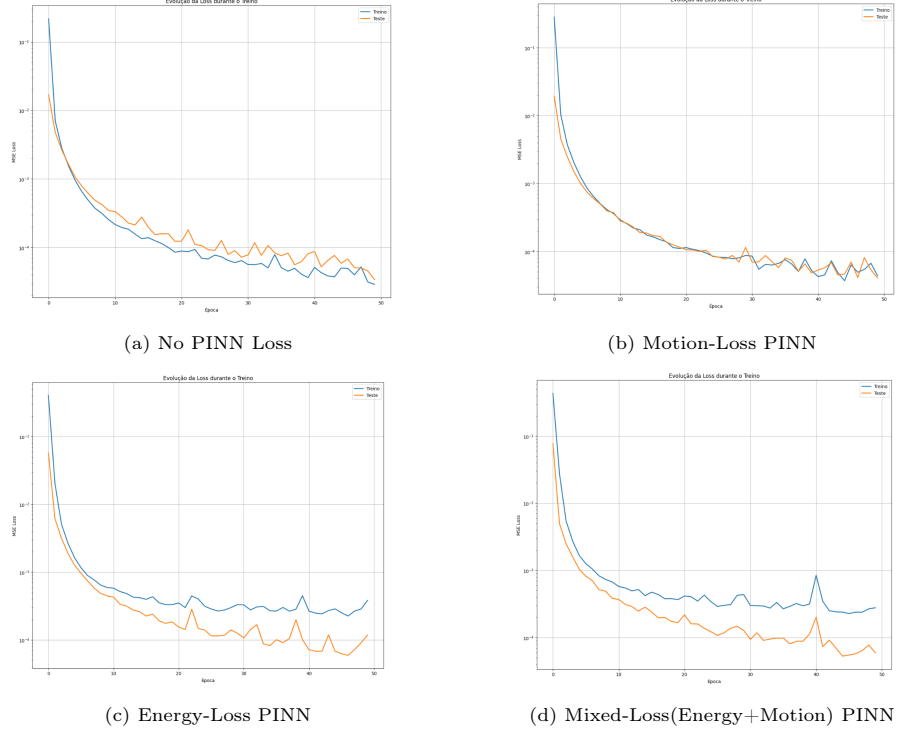


Figure 4: Double pendulum PINN results: physical losses comparison (Motion, Energy e Mixed).

The physical loss tends to dominate over the data loss, therefore the training loss for these PINNs tends to be much higher.

5. Results

There were no significant results with a PINN for the simple pendulum, as the pure LSTM has better results by itself.

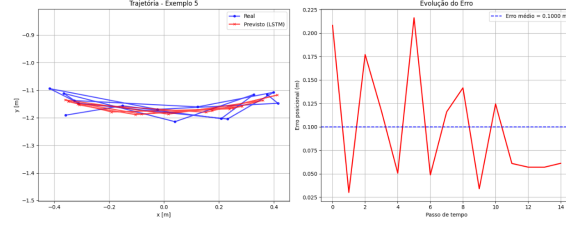


Figure 5: pure LSTM; Mean Error: 0.1000

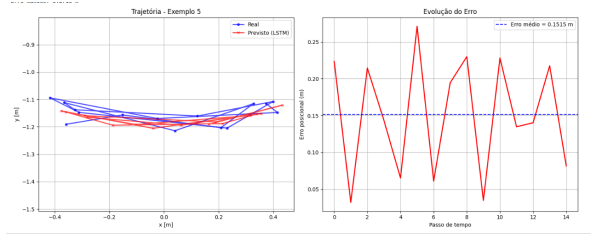


Figure 6: PINN-loss LSTM Mean Error: 0.1515

Now for the double pendulum, the physical loss has a much better outcome:

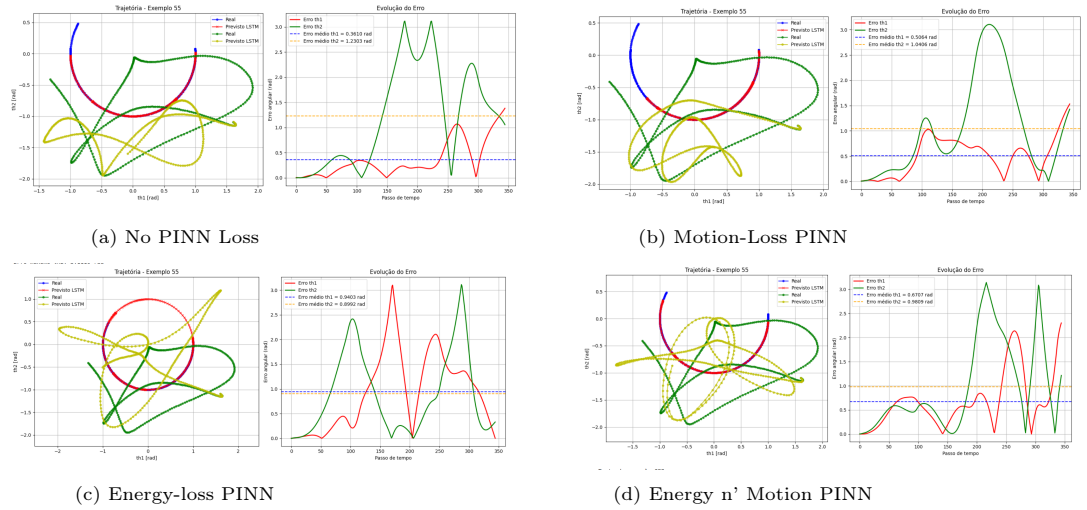


Figure 7: Double pendulum PINN results: physical losses comparison (Motion, Energy and Mixed).

As expected, the results for the simple pendulum are far less prone to error than for the double case.

The values chosen for a model parameters can heavily change it's outcome, time needed for training and general practicality of use. Shown below are some of the results regarding subtle but meaningful changes:

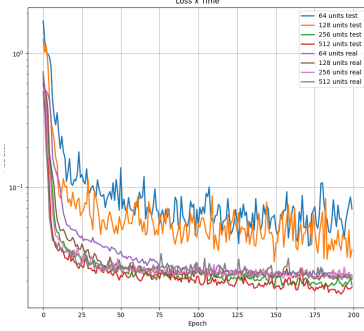


Figure 8: Varying Units

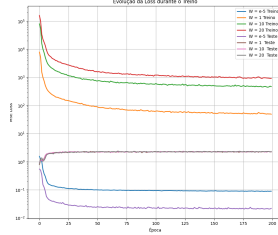


Figure 10: Varying Weights(PINN)

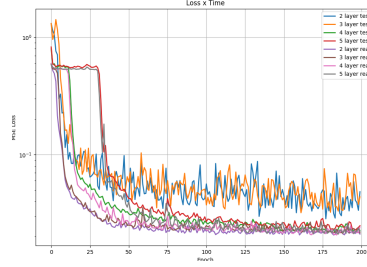


Figure 9: Varying Layers

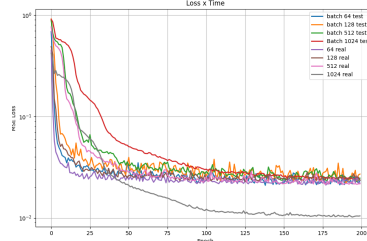


Figure 11: Varying Batch Sizes

Based on the graphics above, it is possible to conclude that the results vary significantly depending on these factors, and the default choice for the models (256 Batch Size, 200 Epochs, 128 hidden units and 2 layers) was made to maximize both precision and accuracy, allowing the authors to test a generous number of cases on their own machines.

6. Conclusion

As seen throughout the project, it is possible to model an adequate LSTM for a simple pendulum with inexpensive resources, without having a substantial loss of accuracy. However, the PINN for this case tends to converge a slower rate, usually resulting in worse predictions.

Meanwhile the prediction of data for a double pendulum seems highly unstable, with poor prediction for both the PINNs and the pure LSTM. However, the PINNs for this case tend to perform slightly better over time.

It is also worth noting that weighted PINNs, with fine-tuned weights, tend to work more efficiently than its unweighted version for this case, as the physical

mean-squared error tends to dominate the training loss, making the network converge very slowly for the major part of the training process.

References

- [1] Dominic Jordan and Peter Smith. *Nonlinear ordinary differential equations*. Oxford Texts in Applied and Engineering Mathematics. Oxford University Press, London, England, 4 edition, August 2007.
- [2] Stephen T Thornton, Jerry B Marion, and All Tasks. *Dinâmica clássica de partículas e sistemas*. 2011.
- [3] R B Levien and S M Tan. Double pendulum: An experiment in chaos. *Am. J. Phys.*, 61(11):1038–1044, November 1993.
- [4] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014.
- [5] Kadierdan Kaheman, Urban Fasel, Jason J. Bramburger, Benjamin Strom, J. Nathan Kutz, and Steven L. Brunton. The experimental multi-arm pendulum on a cart: A benchmark system for chaos, learning, and control. *HardwareX*, 15:e00465, 2023.
- [6] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, March 2020.
- [7] Pawan Goyal and Peter Benner. Discovery of nonlinear dynamical systems using a Runge-Kutta inspired dictionary-based sparse regression approach. *Proc. Math. Phys. Eng. Sci.*, 478(2262):20210883, June 2022.
- [8] Javier Fajardo. Double-pendulum-nn: Physics-informed neural network for the double pendulum, 2025. Accessed: December 7, 2025.
- [9] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.